OAVS EXAM PREP SERIES

Computer Teacher (TGT)

Odisha Adarsha Vidyalaya Sangathan

Qualifiers & Modifiers in C

long · short · signed · unsigned

MCQ PRACTICE BOOK

150+	4	Full	Chapter-
MCQs	Mock Tests	Explanations	wise

Topics Covered in This Book
✔ Type Qualifiers: long, short, signed, unsigned
✔ Memory sizes and byte representations
✔ Ranges for all basic C data types with modifiers
✔ Comparison-based and application MCQs
✔ 4 Full Mock Tests with Answer Keys
✔ Detailed explanations for every question

Prepared for OAVS TGT Examination 2025

Published: May 2026

Chapter TOC: Table of Contents

Chapter 1 — Fundamentals of Type Modifiers in C (Questions 1–20)

Chapter 2 — The 'long' Modifier — Size, Range, Format Specifiers (Questions 21–40)

Chapter 3 — The 'short' Modifier — Memory, Range, Usage (Questions 41–60)

Chapter 4 — The 'signed' Modifier — Representation & Behaviour (Questions 61–80)

Chapter 5 — The 'unsigned' Modifier — Modular Arithmetic & Ranges (Questions 81–100)

Chapter 6 — Comparison & Combined Type Modifier MCQs (Questions 101–125)

Chapter 7 — Advanced & Tricky MCQs (Questions 126–150)

Appendix A — Quick Reference: Data Type Sizes & Ranges Table

Mock Test 1 — Fundamentals & long (25 Questions)

Mock Test 2 — short & signed (25 Questions)

Mock Test 3 — unsigned & Comparison (25 Questions)

Mock Test 4 — Full Mixed Paper (32 Questions)

Answer Keys — All Mock Tests

Chapter : How to Use This Book

Guidance for OAVS TGT Exam Preparation

1. Read each question carefully before looking at the options.
2. Cover the answer/explanation and attempt the question independently.
3. The correct answer is shown in **green bold text**.
4. Study the explanation — many exam questions test conceptual understanding, not just memorisation.
5. Attempt all four Mock Tests under timed conditions (45 minutes each).
6. Focus on memory sizes, format specifiers, and signed/unsigned conversion rules — frequently tested topics.
7. Revise the Quick Reference Table before the exam.

■ Quick Reference: C Data Types — Sizes & Ranges

Data Type	Size (bytes)	Min Value	Max Value	Format Specifier
char	1	-128	127	%c / %d
unsigned char	1	0	255	%c / %u
short int	2	-32,768	32,767	%hd
unsigned short int	2	0	65,535	%hu
int	4	-2,147,483,648	2,147,483,647	%d
unsigned int	4	0	4,294,967,295	%u
long int	4 or 8	-2,147,483,648 (4B)	2,147,483,647 (4B)	%ld
unsigned long int	4 or 8	0	4,294,967,295 (4B)	%lu
long long int	8	-9.2×10^{18}	9.2×10^{18}	%lld
unsigned long long	8	0	1.8×10^{19}	%llu
float	4	$\sim1.2\times10^{-38}$	$\sim3.4\times10^{38}$	%f
double	8	$\sim2.2\times10^{-308}$	$\sim1.8\times10^{308}$	%lf
long double	10 or 16	extended	extended	%Lf

Chapter 1: Fundamentals of Type Modifiers in C

Covers: types of modifiers, validity, defaults, and C standard rules

■ Questions 1–20 with Answers & Explanations

Q1. Which of the following are type modifiers (qualifiers) in C language?

(A) long, short, signed, unsigned

(B) static, extern, register, auto

(C) const, volatile, restrict

(D) int, float, double, char

✔ **Answer: (A)**

Explanation: In C, long, short, signed, and unsigned are type modifiers that alter the range/size of basic data types. The others are storage class specifiers or base data types.

Q2. Which type modifier can be applied to the 'double' data type in C?

(A) short

(B) long

(C) unsigned

(D) Both A and C

✔ **Answer: (B)**

Explanation: 'long double' is valid in C, giving extended precision. 'short double' and 'unsigned double' are NOT valid — floating-point types cannot be unsigned or shortened.

Q3. What is the primary purpose of type modifiers in C?

(A) To change the type of a variable at runtime

(B) To modify the size and range of basic data types

(C) To provide type-casting functionality

(D) To define pointer types

✔ **Answer: (B)**

Explanation: Type modifiers like long, short, signed, and unsigned modify the size (in bytes) and the value range of base data types such as int and char.

Q4. Which of the following declarations is INVALID in C?

(A) unsigned int x;

(B) long double y;

(C) short float z;

(D) signed char c;

✔ **Answer: (C)**

Explanation: 'short float' is not a valid C type. The modifiers short/long cannot be applied to float. Only 'long double' is allowed for floating-point modification.

Q5. The keyword 'signed' in C is used to:

(A) Allow only positive values

(B) Allow both positive and negative values

(C) Double the storage size

(D) Make a variable constant

✔ **Answer: (B)**

Explanation: The 'signed' modifier allows a variable to hold both positive and negative values. By default, int is signed, but explicitly using 'signed' makes the intent clear.

Q6. Which modifier doubles the range of positive values compared to 'signed'?

(A) long

(B) short

(C) unsigned

(D) extern

✔ **Answer: (C)**

Explanation: 'unsigned' removes the sign bit, dedicating all bits to magnitude. Thus an unsigned int (4 bytes) holds 0 to 4,294,967,295 vs –2,147,483,648 to 2,147,483,647 for signed.

Q7. In C, the default sign of 'int' is:

(A) unsigned

(B) signed

(C) depends on the compiler

(D) not applicable

✔ **Answer: (B)**

Explanation: By the C standard, 'int' without any sign modifier defaults to 'signed int'. However, 'char' sign is implementation-defined.

Q8. Which of the following is NOT a valid use of 'long' modifier?

(A) long int

(B) long double

(C) long char

(D) long long int

✔ **Answer: (C)**

Explanation: 'long char' is invalid in C. The 'long' modifier can be applied to int (long int), double (long double), and can be doubled (long long int).

Q9. What is the size of 'short int' on most 32-bit and 64-bit systems?

(A) 1 byte

(B) 2 bytes

(C) 4 bytes

(D) 8 bytes

✔ **Answer: (B)**

Explanation: 'short int' is guaranteed by the C standard to be at least 2 bytes (16 bits). On virtually all modern platforms it is exactly 2 bytes.

Q10. Which C standard guarantees that sizeof(short) ≤ sizeof(int) ≤ sizeof(long)?

(A) ANSI C (C89/C90)

(B) C99

(C) C11

(D) All C standards

✔ **Answer: (D)**

Explanation: All C standards (C89 onwards) mandate the relationship sizeof(short) ≤ sizeof(int) ≤ sizeof(long), ensuring a consistent hierarchy of integer sizes.

Q11. Which modifier is used to extend the range of an integer by using more storage?

(A) short

(B) unsigned

(C) long

(D) register

✔ **Answer: (C)**

Explanation: 'long' extends the storage (and thus range) of an integer. 'long int' is at least 4 bytes, and 'long long int' is at least 8 bytes.

Q12. Type modifier 'unsigned' can be applied to which of the following?

(A) float and double

(B) int and char only

(C) int, char, short, long

(D) All data types

✔ **Answer: (C)**

Explanation: 'unsigned' applies only to integer types: char, short, int, long, and long long. It cannot be applied to floating-point types.

Q13. Which of the following correctly declares a 'short unsigned integer'?

(A) short unsigned int x;

(B) unsigned short x;

(C) Both A and B

(D) Neither A nor B

✔ **Answer: (C)**

Explanation: Both 'short unsigned int x;' and 'unsigned short x;' are valid and equivalent declarations. C allows modifiers in any order.

Q14. What does the 'signed' keyword do when applied to 'char'?

(A) Makes char hold –128 to 127

(B) Makes char hold 0 to 255

(C) Makes char hold –256 to 255

(D) No effect on char

✔ **Answer: (A)**

Explanation: 'signed char' is guaranteed to hold values from –128 to 127 (8-bit two's complement). This is important because plain 'char' sign is implementation-defined.

Q15. The 'unsigned char' type in C has a range of:

(A) –128 to 127

(B) 0 to 127

(C) 0 to 255

(D) –256 to 255

✔ **Answer: (C)**

Explanation: 'unsigned char' uses all 8 bits for magnitude (no sign bit), giving values from 0 to 255 ($2^8 – 1 = 255$).

Q16. What is the minimum size (in bytes) guaranteed by the C standard for 'long long int'?

(A) 4

(B) 6

(C) 8

(D) 16

✔ **Answer: (C)**

Explanation: The C99 standard introduced 'long long int' with a guaranteed minimum of 8 bytes (64 bits), providing at least the range –9,223,372,036,854,775,808 to +9,223,372,036,854,775,807.

Q17. Which header file provides the exact-width integer types like int8_t, uint32_t?

(A)

(B)

(C)

(D)

✔ **Answer: (C)**

Explanation: (C99) defines exact-width integer types (int8_t, uint32_t, etc.), minimum-width types, and fastest types, ensuring portability.

Q18. Which macro in gives the maximum value of 'unsigned int'?

(A) INT_MAX

(B) UINT_MAX

(C) ULONG_MAX

(D) SHORT_MAX

✔ **Answer: (B)**

Explanation: UINT_MAX in holds the maximum value of unsigned int. For a 4-byte unsigned int this is 4,294,967,295 ($2^{32} – 1$).

Q19. Which of the following statements about type modifiers is TRUE?

(A) 'short' always means 2 bytes on all platforms

(B) 'long' always means 8 bytes on all platforms

(C) sizeof(int) is guaranteed to be 4 bytes

(D) sizeof(short) ≤ sizeof(long) is always guaranteed

✔ **Answer: (D)**

Explanation: The C standard only guarantees sizeof(short) ≤ sizeof(int) ≤ sizeof(long). Exact sizes are platform/compiler-dependent (though short=2 and int=4 are nearly universal today).

Q20. When 'long' is written alone without 'int', it is treated as:

(A) long double

(B) long float

(C) long int

(D) Compiler error

✔ **Answer: (C)**

Explanation: In C, 'long' written alone (e.g., 'long x;') is shorthand for 'long int'. The 'int' keyword is optional when a modifier is present.

Chapter 2: The 'long' Modifier

Memory sizes, ranges, format specifiers, data models, and literals

■ Questions 21–40 with Answers & Explanations

Q21. What is the typical size of 'long int' on a 32-bit compiler?

(A) 2 bytes

(B) 4 bytes

(C) 8 bytes

(D) 16 bytes

✔ **Answer: (B)**

Explanation: On most 32-bit compilers (e.g., MSVC on Windows), 'long int' is 4 bytes. On 64-bit Linux/macOS (LP64 model) it is 8 bytes.

Q22. What is the size of 'long int' on a 64-bit Linux system (GCC)?

(A) 4 bytes

(B) 6 bytes

(C) 8 bytes

(D) 16 bytes

✔ **Answer: (C)**

Explanation: On 64-bit Linux using the LP64 data model (GCC), 'long int' is 8 bytes. On 64-bit Windows (LLP64 model), it remains 4 bytes.

Q23. What is the format specifier for 'long int' in C?

(A) %d

(B) %ld

(C) %li

(D) Both B and C

✔ **Answer: (D)**

Explanation: Both %ld and %li are valid format specifiers for 'long int' with printf/scanf. %ld is more commonly used.

Q24. What is the range of 'long int' when it is 4 bytes?

(A) –32,768 to 32,767

(B) –2,147,483,648 to 2,147,483,647

(C) 0 to 4,294,967,295

(D) –9,223,372,036,854,775,808 to 9,223,372,036,854,775,807

✔ **Answer: (B)**

Explanation: A 4-byte signed long int uses 32 bits in two's complement: -2^{31} = –2,147,483,648 to $2^{31}-1$ = 2,147,483,647.

Q25. What is the range of 'long int' when it is 8 bytes (64-bit)?

(A) 0 to 18,446,744,073,709,551,615

(B) –2,147,483,648 to 2,147,483,647

(C) –9,223,372,036,854,775,808 to 9,223,372,036,854,775,807

(D) –4,294,967,296 to 4,294,967,295

✔ **Answer: (C)**

Explanation: An 8-byte signed long int uses 64 bits: -2^{63} to $2^{63}-1$, i.e., approximately -9.2×10^{18} to 9.2×10^{18}.

Q26. What does 'long long int' guarantee in terms of size?

(A) At least 4 bytes

(B) At least 6 bytes

(C) Exactly 8 bytes

(D) At least 8 bytes

✔ **Answer: (D)**

Explanation: The C99 standard guarantees 'long long int' is at least 8 bytes (64 bits). On all current implementations it is exactly 8 bytes.

Q27. Which format specifier is used for 'long long int'?

(A) %ld

(B) %llx

(C) %lld

(D) %Ld

✔ **Answer: (C)**

Explanation: %lld (two 'l's followed by 'd') is the correct format specifier for 'long long int'. %llx is for long long in hexadecimal. %Ld is not standard.

Q28. What is the format specifier for 'unsigned long int'?

(A) %u

(B) %lu

(C) %ul

(D) %ld

✔ **Answer: (B)**

Explanation: %lu is the format specifier for 'unsigned long int'. %u is for plain 'unsigned int'. Mixing these may produce warnings or wrong output.

Q29. Evaluate: What will sizeof(long long int) return on a 32-bit GCC compiler?

(A) 4

(B) 6

(C) 8

(D) 16

✔ **Answer: (C)**

Explanation: Even on 32-bit GCC, 'long long int' is always 8 bytes as mandated by C99. It is the only way in C89/C90 code to guarantee 64-bit storage without using stdint.h.

Q30. Which suffix is used to denote a long integer literal in C?

(A) L or l

(B) U or u

(C) LL or ll

(D) LU or lu

✔ **Answer: (A)**

Explanation: The suffix 'L' or 'l' denotes a long integer literal (e.g., 100L). 'LL' or 'll' denotes long long. 'U'/'u' denotes unsigned.

Q31. Which suffix is used to denote a long long integer literal?

(A) L

(B) ll

(C) LL or ll

(D) LLL

✔ **Answer: (C)**

Explanation: 'LL' or 'll' (case-insensitive pair) is the suffix for long long integer literals. Example: 9000000000LL.

Q32. The LLONG_MAX macro represents the maximum value of 'long long int'. Its value is approximately:

(A) 2.1 billion

(B) 4.3 billion

(C) 9.2×10^{18}

(D) 1.8×10^{19}

✔ **Answer: (C)**

Explanation: LLONG_MAX = $2^{63}-1 = 9{,}223{,}372{,}036{,}854{,}775{,}807 \approx 9.2\times10^{18}$. ULLONG_MAX = $2^{64}-1 \approx 1.8\times10^{19}$.

Q33. Which C standard introduced 'long long int'?

(A) ANSI C (C89)

(B) C95

(C) C99

(D) C11

✔ **Answer: (C)**

Explanation: 'long long int' was officially introduced in C99 (ISO/IEC 9899:1999). Though many compilers supported it as an extension before C99.

Q34. What is printed by: printf("%ld", 2147483648L); on a system where long is 4 bytes?

(A) 2147483648

(B) -2147483648

(C) 0

(D) Undefined behaviour

✔ **Answer: (D)**

Explanation: 2,147,483,648 exceeds INT_MAX (2,147,483,647) for a 4-byte long. The value overflows a 4-byte signed long, resulting in undefined behaviour.

Q35. If sizeof(long) = 8, what is LONG_MAX?

(A) 2,147,483,647

(B) 4,294,967,295

(C) 9,223,372,036,854,775,807

(D) 18,446,744,073,709,551,615

✔ **Answer: (C)**

Explanation: For an 8-byte (64-bit) signed long, LONG_MAX = 2^{63}–1 = 9,223,372,036,854,775,807.

Q36. Which of the following is the correct way to read a long int using scanf?

(A) scanf("%d", &x;);

(B) scanf("%ld", &x;);

(C) scanf("%i", &x;);

(D) scanf("%li", &x;);

✔ **Answer: (B)**

Explanation: scanf("%ld", &x;) is the correct way for long int. Using %d for a long int leads to undefined behaviour on platforms where sizeof(long) ≠ sizeof(int).

Q37. Which data model uses long = 8 bytes and int = 4 bytes?

(A) ILP32

(B) LP64

(C) LLP64

(D) ILP64

✔ **Answer: (B)**

Explanation: LP64 (Long-Pointer 64) is used on 64-bit Linux/macOS: int=4B, long=8B, pointer=8B. LLP64 (Windows 64-bit): int=4B, long=4B, long long=8B, pointer=8B.

Q38. In which data model is 'long' still 4 bytes even on a 64-bit OS?

(A) LP64

(B) ILP64

(C) LLP64 (Windows 64-bit)

(D) ILP32

✔ **Answer: (C)**

Explanation: Windows 64-bit uses the LLP64 model where long is 4 bytes. Only 'long long' and pointers are 8 bytes. This is a common portability trap.

Q39. What is the output of: printf("%d", sizeof(long)); on a typical 64-bit Linux?

(A) 2

(B) 4

(C) 8

(D) 16

✔ **Answer: (C)**

Explanation: On 64-bit Linux (GCC, LP64 model), sizeof(long) = 8. Printing sizeof results with %d is technically wrong (%zu is correct), but returns 8 here.

Q40. Which is the correct portable way to print sizeof() results?

(A) %d

(B) %i

(C) %u

(D) %zu

✔ **Answer: (D)**

Explanation: %zu is the correct format specifier for size_t (the type returned by sizeof). Using %d or %u may cause warnings or incorrect output on 64-bit platforms.

Chapter 3: The 'short' Modifier

2-byte integers, SHRT_MAX, SHRT_MIN, practical applications

■ Questions 41–60 with Answers & Explanations

Q41. A 'long double' typically provides how many bytes of storage in GCC on x86-64 Linux?

(A) 8 bytes
(B) 10 bytes
(C) 12 bytes
(D) 16 bytes (80-bit extended stored in 16B)

✔ **Answer: (D)**
Explanation: On GCC x86-64 Linux, 'long double' uses the 80-bit x87 extended precision, stored in 16 bytes (with padding) due to alignment. sizeof(long double) = 16.

Q42. What is the size of 'short int' in C?

(A) 1 byte
(B) 2 bytes
(C) 4 bytes
(D) Depends only on OS

✔ **Answer: (B)**
Explanation: The C standard mandates sizeof(short) ≥ 1 and sizeof(short) ≤ sizeof(int). In practice, short int is universally 2 bytes (16 bits).

Q43. What is the range of 'signed short int' (2 bytes)?

(A) 0 to 65,535
(B) –128 to 127
(C) –32,768 to 32,767
(D) –2,147,483,648 to 2,147,483,647

✔ **Answer: (C)**
Explanation: A 2-byte signed short uses 16 bits: -2^{15} = –32,768 to $2^{15}-1$ = 32,767.

Q44. What is the range of 'unsigned short int' (2 bytes)?

(A) –32,768 to 32,767
(B) 0 to 32,767
(C) 0 to 65,535
(D) 0 to 65,536

✔ **Answer: (C)**
Explanation: An unsigned short int (2 bytes) ranges from 0 to $2^{16}-1$ = 65,535. All 16 bits are used for magnitude.

Q45. The format specifier for 'short int' in printf is:

(A) %d

(B) %hd

(C) %sd

(D) %i

✔ **Answer: (B)**

Explanation: %hd is the correct format specifier for 'short int'. 'h' is the length modifier indicating a short argument (though in printf, short is promoted to int).

Q46. The format specifier for 'unsigned short int' is:

(A) %u

(B) %hu

(C) %su

(D) %uh

✔ **Answer: (B)**

Explanation: %hu is the format specifier for 'unsigned short int'. For scanf it is critical; for printf, short is auto-promoted to int/unsigned int anyway.

Q47. Which macro holds the maximum value of 'short int'?

(A) SHORT_MAX

(B) SHRT_MAX

(C) MAX_SHORT

(D) INT16_MAX

✔ **Answer: (B)**

Explanation: SHRT_MAX is defined in and equals 32,767 for a 2-byte short. INT16_MAX (from) is equivalent (32,767) for exact-width types.

Q48. Which macro holds the minimum value of 'short int'?

(A) SHRT_MIN

(B) SHORT_MIN

(C) INT16_MIN

(D) Both A and C

✔ **Answer: (D)**

Explanation: SHRT_MIN (in) = –32,768 for 2-byte short. INT16_MIN (in) = –32,768 for int16_t. Both represent the same value.

Q49. Why is 'short int' used instead of 'int' in some applications?

(A) It provides higher precision

(B) It saves memory in large arrays

(C) It is faster on all CPUs

(D) It allows negative values

✔ **Answer: (B)**

Explanation: short int (2 bytes) saves memory compared to int (4 bytes). When millions of integers are stored (e.g., image pixels, audio samples), using short can halve memory usage.

Q50. What happens when a short int value exceeds 32,767 (e.g., you assign 32768)?

(A) It becomes 32768

(B) It wraps around to –32,768

(C) It becomes 0

(D) Runtime error

✔ **Answer: (B)**

Explanation: Signed integer overflow is undefined behaviour in C, but in practice (two's complement), 32,768 wraps to –32,768 because the bit pattern 0x8000 represents –32,768 in signed 16-bit.

Q51. What is the result of: short s = 32767; s++; (assuming two's complement)?

(A) 32768

(B) –32768

(C) 0

(D) Undefined, then likely –32768

✔ **Answer: (D)**

Explanation: Signed overflow is technically undefined behaviour in C. In practice on two's complement systems, it wraps to –32,768. The correct answer acknowledges both the UB aspect and the practical result.

Q52. Which of the following is a valid declaration of a short integer?

(A) short x;

(B) short int x;

(C) int short x;

(D) All of the above

✔ **Answer: (D)**

Explanation: All three are valid C declarations. 'short', 'short int', and 'int short' all declare a short integer. C allows modifiers in flexible order.

Q53. What is sizeof(short) + sizeof(int) on a typical 32-bit system?

(A) 4

(B) 6

(C) 8

(D) Compiler-dependent

✔ **Answer: (B)**

Explanation: On a typical 32-bit system, sizeof(short)=2 and sizeof(int)=4, so 2+4=6. This is nearly universal, though strictly compiler-dependent.

Q54. In which scenario would 'short int' be preferred over 'int'?

(A) When you need values beyond 32,767

(B) When declaring a single counter variable

(C) When storing large arrays of small numbers to save memory

(D) When writing to a long-term file

✔ **Answer: (C)**

Explanation: short int is preferred for large arrays of small-range integers (like 16-bit audio samples or pixel values) where memory efficiency matters.

Q55. What is the size of 'unsigned short' in bytes?

(A) 1

(B) 2

(C) 4

(D) Same as int

✔ **Answer: (B)**

Explanation: unsigned short is 2 bytes, same as short int. The 'unsigned' modifier does not change the size, only the interpretation of the sign bit.

Q56. Which of the following represents a 16-bit port number correctly in C?

(A) int port = 8080;

(B) short port = 8080;

(C) unsigned short port = 8080;

(D) char port = 8080;

✔ **Answer: (C)**

Explanation: Network port numbers range from 0 to 65,535, fitting exactly in an unsigned short (2 bytes, 0–65,535). Using 'unsigned short' is semantically correct and common in networking code.

Q57. USHRT_MAX in is equal to:

(A) 32,767

(B) 65,535

(C) 65,536

(D) 32,768

✔ **Answer: (B)**

Explanation: USHRT_MAX = 2^{16}–1 = 65,535 for unsigned short int (2 bytes).

Q58. When a short int is passed to printf with %d, what conversion occurs?

(A) Nothing, %d works directly

(B) Integer promotion to int happens automatically

(C) The value is truncated to 1 byte

(D) Undefined behaviour always occurs

✔ **Answer: (B)**

Explanation: In C, when passing short to printf, integer promotion occurs automatically — the short is promoted to int. This is why %d works fine for short int in printf.

Q59. What is the two's complement binary representation of –1 in a 16-bit short?

(A) 0111 1111 1111 1111

(B) 1000 0000 0000 0001

(C) 1111 1111 1111 1111

(D) 1000 0000 0000 0000

✔ **Answer: (C)**

Explanation: In two's complement, –1 is represented as all 1s: 1111 1111 1111 1111 (0xFFFF) for 16-bit. This is a fundamental property of two's complement arithmetic.

Q60. What is the binary representation of SHRT_MAX (32,767) in 16 bits?

(A) 0111 1111 1111 1111

(B) 1111 1111 1111 1111

(C) 0111 1111 0000 0000

(D) 1000 0000 0000 0000

✔ **Answer: (A)**

Explanation: 32,767 = 2^{15}–1 = 0x7FFF = 0111 1111 1111 1111 in 16-bit binary. The MSB (sign bit) is 0, indicating positive.

Chapter 4: The 'signed' Modifier

Two's complement, sign bit, INT_MAX/INT_MIN, signed behaviour

■ Questions 61–80 with Answers & Explanations

Q61. What is SHRT_MIN, the minimum value of short int?

(A) 0

(B) –32,767

(C) –32,768

(D) –65,536

✔ **Answer: (C)**

Explanation: SHRT_MIN = -2^{15} = –32,768 in two's complement 16-bit representation (bit pattern: 1000 0000 0000 0000 = 0x8000).

Q62. Which of the following is true about the 'signed' modifier?

(A) It is the default for all integer types

(B) It allows values from 0 to 2n-1

(C) It allows both positive and negative values

(D) It only applies to char

✔ **Answer: (C)**

Explanation: 'signed' allows variables to hold both positive and negative values using the MSB as a sign bit. It is the default for int, short, and long (but not char — char's sign is implementation-defined).

Q63. What is the range of 'signed char'?

(A) 0 to 255

(B) –128 to 127

(C) –127 to 127

(D) –256 to 255

✔ **Answer: (B)**

Explanation: signed char uses 8 bits: 1 sign bit + 7 value bits. In two's complement: -2^7 = –128 to 2^7-1 = 127.

Q64. The two's complement representation of –128 in a signed char (8 bits) is:

(A) 0111 1111

(B) 1111 1111

(C) 1000 0000

(D) 1000 0001

✔ **Answer: (C)**

Explanation: –128 in 8-bit two's complement is 1000 0000 (0x80). This is the only 8-bit pattern without a positive counterpart in two's complement.

Q65. What is the output of the following code? signed int x = -1; printf("%u", x);

(A) –1

(B) 4294967295

(C) 0

(D) Compiler error

✔ **Answer: (B)**

Explanation: –1 in 32-bit two's complement is 0xFFFFFFFF. When reinterpreted as unsigned (%u), it prints 4,294,967,295 (2^{32}–1). This is common in C bit-manipulation.

Q66. For signed integer types, how is negative zero handled in C?

(A) It is a valid value distinct from zero

(B) Two's complement has no negative zero

(C) It is defined as –0 in the standard

(D) It causes undefined behaviour

✔ **Answer: (B)**

Explanation: Two's complement (used universally in modern CPUs) has no negative zero. –0 and 0 have the same bit pattern. The C standard mandates two's complement since C20.

Q67. CHAR_MIN in will be –128 if:

(A) char is unsigned

(B) char is signed

(C) char is 16-bit

(D) char is long

✔ **Answer: (B)**

Explanation: CHAR_MIN = –128 when char is signed (signed char). If char is unsigned, CHAR_MIN = 0. The sign of plain char is implementation-defined.

Q68. Which of the following produces correct results when checking if c is a valid ASCII character?

(A) if (c >= 0 && c < 128) — assuming char is signed

(B) if ((unsigned char)c < 128)

(C) Both A and B are correct

(D) Neither is correct

✔ **Answer: (C)**

Explanation: Both work for ASCII (0–127). However, (B) using unsigned char cast is safer and portable. ASCII values are always non-negative, so both conditions are equivalent for valid ASCII.

Q69. What will happen if –129 is assigned to a signed char?

(A) The value is stored as –129

(B) Wraps around: –129 becomes 127

(C) Wraps around: –129 becomes –1

(D) Undefined/implementation-defined overflow

✔ **Answer: (D)**

Explanation: Assigning an out-of-range value to a signed char is implementation-defined in C (not undefined, but not predictable without knowing the implementation).

Q70. What does 'int' default to if no sign modifier is written?

(A) unsigned

(B) signed

(C) depends on the value assigned

(D) compiler chooses randomly

✔ **Answer: (B)**

Explanation: By the C standard, 'int' without a sign modifier defaults to 'signed int'. This is guaranteed — unlike plain 'char' whose sign is implementation-defined.

Q71. Which bit is the sign bit in a signed representation?

(A) Least Significant Bit (LSB)

(B) Most Significant Bit (MSB)

(C) Middle bit

(D) Second LSB

✔ **Answer: (B)**

Explanation: In signed integer representations, the MSB (Most Significant Bit / highest-order bit) is the sign bit: 0 = positive, 1 = negative (in two's complement).

Q72. The expression (signed char)(200) on a typical system gives:

(A) 200

(B) –56

(C) 56

(D) 0

✔ **Answer: (B)**

Explanation: 200 in binary is 1100 1000. As a signed char (8-bit two's complement), MSB=1 so it's negative. Two's complement of 1100 1000 → invert → 0011 0111 → add 1 → 0011 1000 = 56. So the value is –56.

Q73. What is the value of INT_MAX on a system where sizeof(int) = 4?

(A) 32,767

(B) 65,535

(C) 2,147,483,647

(D) 4,294,967,295

✔ **Answer: (C)**

Explanation: INT_MAX = 2^{31}–1 = 2,147,483,647 for a 4-byte signed int. This is the maximum positive value of a 32-bit signed integer.

Q74. What is INT_MIN on a 4-byte signed int system?

(A) –2,147,483,647

(B) –2,147,483,648

(C) –4,294,967,295

(D) –2,147,483,649

✔ **Answer: (B)**

Explanation: INT_MIN = $–2^{31}$ = –2,147,483,648. Note the asymmetry: |INT_MIN| = |INT_MAX| + 1 due to two's complement, which has one extra negative value.

Q75. Which of the following will cause signed integer overflow (undefined behaviour)?

(A) int x = INT_MAX; x++;

(B) unsigned int x = UINT_MAX; x++;

(C) short x = 32767; x = (short)(x+1);

(D) Both A and C

✔ **Answer: (A)**

Explanation: Signed integer overflow (INT_MAX + 1) is undefined behaviour in C. Unsigned overflow (UINT_MAX + 1 → 0) is well-defined (wraps). (C) is implementation-defined, not strictly UB.

Q76. What arithmetic is used to represent negative numbers in virtually all modern CPUs?

(A) Sign-magnitude

(B) One's complement

(C) Two's complement

(D) BCD

✔ **Answer: (C)**

Explanation: Two's complement is used by virtually all modern processors. C20 mandates it. Advantages: unique zero, simpler hardware addition, and clean wraparound for unsigned.

Q77. In C, which of the following is a signed data type by default?

(A) char

(B) int

(C) Both A and B

(D) Neither — both are unsigned by default

✔ **Answer: (B)**

Explanation: 'int' is always signed by default. 'char' is implementation-defined (could be signed or unsigned). So strictly, only 'int' is guaranteed signed without an explicit modifier.

Q78. When you negate INT_MIN in C signed arithmetic, what happens?

(A) You get INT_MAX

(B) You get –INT_MIN = INT_MIN (overflow!)

(C) You get 0

(D) Compiler error

✔ **Answer: (B)**

Explanation: –INT_MIN cannot be represented as a signed int (since |INT_MIN| > INT_MAX). The operation overflows, which is undefined behaviour. In practice, –(–2147483648) overflows back to –2147483648.

Q79. What is printed by: printf("%d", -1 >> 1); on most systems?

(A) –1

(B) 2147483647

(C) 0

(D) Undefined

✔ **Answer: (A)**

Explanation: Right-shifting a signed negative value is implementation-defined. On most (arithmetic shift) systems, –1 >> 1 = –1 because the sign bit is propagated. This is why it's called arithmetic right shift.

Q80. Which function converts a string to a signed long integer?

(A) atoi()

(B) strtoul()

(C) strtol()

(D) scanf() only

✔ **Answer: (C)**

Explanation: strtol() (string-to-long) converts a string to a signed long int with error handling. atoi() converts to int. strtoul() converts to unsigned long.

Chapter 5: The 'unsigned' Modifier

Unsigned ranges, modular arithmetic, bitwise operations, pitfalls

■ Questions 81–100 with Answers & Explanations

Q81. Which header provides the abs() function for signed int?

(A)

(B)

(C)

(D)

✔ **Answer: (B)**

Explanation: abs() for int is in . labs() for long and llabs() for long long are also in . fabs() for floating-point is in .

Q82. What does labs() do in C?

(A) Absolute value of long int

(B) Absolute value of float

(C) Logarithm of long

(D) Load absolute symbol

✔ **Answer: (A)**

Explanation: labs() (defined in) returns the absolute value of a long int. It is the long int version of abs(). llabs() handles long long int.

Q83. What is the range of 'unsigned int' when sizeof(int) = 4?

(A) –2,147,483,648 to 2,147,483,647

(B) 0 to 2,147,483,647

(C) 0 to 4,294,967,295

(D) –4,294,967,296 to 4,294,967,295

✔ **Answer: (C)**

Explanation: unsigned int (4 bytes, 32 bits): all bits used for magnitude. Range: 0 to 2^{32}–1 = 4,294,967,295 (UINT_MAX).

Q84. What format specifier is used for 'unsigned int'?

(A) %d

(B) %u

(C) %i

(D) %ud

✔ **Answer: (B)**

Explanation: %u is the format specifier for unsigned int. Using %d for an unsigned int may print negative values if the MSB is 1.

Q85. What is the output of: unsigned int x = 0; x--; printf("%u", x);

(A) –1

(B) 0

(C) 4294967295

(D) Undefined behaviour

✔ **Answer: (C)**

Explanation: Unsigned integer arithmetic wraps modulo 2^n. 0 – 1 = –1 ≡ 4,294,967,295 (mod 2^{32}). Unsigned underflow/overflow is well-defined in C.

Q86. What is the value of UINT_MAX on a 4-byte unsigned int system?

(A) 2,147,483,647

(B) 4,294,967,295

(C) 4,294,967,296

(D) 2,147,483,648

✔ **Answer: (B)**

Explanation: UINT_MAX = 2^{32}–1 = 4,294,967,295. It is defined in .

Q87. Which suffix denotes an unsigned integer literal in C?

(A) L

(B) U or u

(C) S

(D) UL

✔ **Answer: (B)**

Explanation: 'U' or 'u' suffix makes an integer literal unsigned (e.g., 100U). 'UL' or 'LU' makes it unsigned long. 'ULL' makes it unsigned long long.

Q88. What is the result of: (unsigned int)(–1) on a 4-byte int system?

(A) –1

(B) 0

(C) 4294967295

(D) Compiler error

✔ **Answer: (C)**

Explanation: Converting –1 (signed) to unsigned int: the bit pattern 0xFFFFFFFF is reinterpreted as 4,294,967,295. This is well-defined conversion in C.

Q89. Unsigned arithmetic overflow in C is:

(A) Undefined behaviour

(B) Implementation-defined

(C) Well-defined (wraps modulo 2n)

(D) Always an error

✔ **Answer: (C)**

Explanation: Unsigned arithmetic overflow is well-defined in C: it wraps modulo 2^n where n is the width. UINT_MAX + 1 = 0, 0 – 1 = UINT_MAX.

Q90. What is the output of: printf("%u", UINT_MAX + 1U); on a 4-byte int system?

(A) 4294967296

(B) 4294967295

(C) 0

(D) Undefined

✔ **Answer: (C)**

Explanation: UINT_MAX + 1U wraps to 0 due to unsigned modular arithmetic (2^{32} mod 2^{32} = 0). This is well-defined behaviour.

Q91. Which of the following is used to store a memory address or object size safely?

(A) unsigned int

(B) int

(C) size_t

(D) long int

✔ **Answer: (C)**

Explanation: size_t (defined in) is an unsigned integer type guaranteed to hold the size of any object in memory. On 64-bit systems it is 8 bytes, on 32-bit systems 4 bytes.

Q92. What is the range of 'unsigned char' (1 byte)?

(A) 0 to 127

(B) –128 to 127

(C) 0 to 255

(D) 0 to 256

✔ **Answer: (C)**

Explanation: unsigned char uses all 8 bits for magnitude: 0 to 2^8–1 = 255.

Q93. What is the range of 'unsigned short int' (2 bytes)?

(A) 0 to 32,767

(B) 0 to 65,535

(C) –32,768 to 32,767

(D) 0 to 65,536

✔ **Answer: (B)**

Explanation: unsigned short int: 0 to 2^{16}–1 = 65,535. Common usage: port numbers, pixel component values, UTF-16 code units.

Q94. What is the range of 'unsigned long long int' (8 bytes)?

(A) 0 to 2^{63}–1

(B) 0 to 2^{64}–1

(C) -2^{63} to 2^{63}–1

(D) 0 to 2^{64}

✔ **Answer: (B)**

Explanation: unsigned long long (8 bytes, 64 bits): 0 to 2^{64}–1 = 18,446,744,073,709,551,615 (ULLONG_MAX).

Q95. Why should unsigned types be used for bit manipulation?

(A) They are faster

(B) They avoid undefined behaviour from sign-bit shifts

(C) They use less memory

(D) They support all arithmetic operators

✔ **Answer: (B)**

Explanation: Right-shifting a negative signed value is implementation-defined, and left-shifting into the sign bit is undefined. Using unsigned types makes bit manipulation well-defined and portable.

Q96. In a comparison 'if (a > b)' where a is signed int (–1) and b is unsigned int (0), what happens?

(A) –1 > 0 is false, as expected

(B) –1 is converted to unsigned (4294967295), so 4294967295 > 0 is true

(C) Compiler rejects mixed comparison

(D) Both are converted to float

✔ **Answer: (B)**

Explanation: Signed–unsigned comparison: the signed value is converted to unsigned. –1 (signed int) becomes 4,294,967,295 (unsigned). So 4,294,967,295 > 0 is TRUE — a classic bug in C.

Q97. What is the result of: unsigned int x = 5; int y = –10; if (x > y)? (Hint: signed/unsigned comparison rules apply)

(A) Branch NOT taken (–10 converts to ~4.3 billion as unsigned, so 5 < 4294967286)

(B) Branch taken (5 > –10 mathematically)

(C) Compiler error: cannot compare signed and unsigned

(D) Undefined behaviour

✔ **Answer: (A)**

Explanation: In C, when comparing signed int and unsigned int, the signed value is converted to unsigned. –10 becomes 4,294,967,286 (a huge number). So 5 > 4,294,967,286 is FALSE. The branch is NOT taken. This is a notorious C gotcha!

Q98. Which of these avoids the signed/unsigned comparison warning?

(A) if (i > (signed)size)

(B) if ((size_t)i > size)

(C) if (i > size)

(D) if (i – size > 0)

✔ **Answer: (B)**

Explanation: Casting i to size_t (unsigned) before comparison eliminates the signed/unsigned mismatch, provided i is known to be non-negative. This is the idiomatic solution.

Q99. What is the result of: unsigned int x = 5; int y = –10; if(x > y) ...

(A) Branch taken (5 > –10 is mathematically true)

(B) Branch NOT taken (–10 converts to large unsigned)

(C) Compiler error

(D) Depends on CPU endianness

✔ **Answer: (A)**

Explanation: Wait — actually: in C, when comparing unsigned int and int, int is converted to unsigned. –10 → 4,294,967,286 (unsigned). So 5 > 4,294,967,286 is FALSE. The branch is NOT taken. Correct

answer is B.

Q100. The suffix 'ULL' in C represents:

(A) unsigned long

(B) unsigned long long

(C) unsigned long literal

(D) Union-linked list

✔ **Answer: (B)**

Explanation: 'ULL' (or 'ull') suffix denotes an unsigned long long integer literal. Example: 18446744073709551615ULL = ULLONG_MAX.

Chapter 6: Comparison & Combined MCQs

Contrasting modifiers, choosing the right type, practical scenarios

■ Questions 101–125 with Answers & Explanations

Q101. The format specifier for 'unsigned long long int' is:

(A) %lu

(B) %llu

(C) %uLL

(D) %Llu

✔ **Answer: (B)**

Explanation: %llu is the format specifier for unsigned long long int (two 'l's then 'u'). Incorrect specifiers cause undefined behaviour in printf/scanf.

Q102. Which of the following detects if an unsigned value has a specific bit set?

(A) if (x AND bit)

(B) if (x & bit)

(C) if (x | bit)

(D) if (x ^ bit)

✔ **Answer: (B)**

Explanation: Bitwise AND (&) tests specific bits. 'if (x & (1U << n))' checks if bit n is set. This works cleanly with unsigned types.

Q103. What is the output of: unsigned char c = 255; c++; printf("%d", c);

(A) 256

(B) –1

(C) 0

(D) 255

✔ **Answer: (C)**

Explanation: unsigned char overflows modulo 2^8 = 256. So 255 + 1 = 256 mod 256 = 0. Well-defined wraparound for unsigned types.

Q104. Which modifier pair represents opposite concepts in C type system?

(A) long and short

(B) signed and unsigned

(C) Both A and B

(D) Neither

✔ **Answer: (C)**

Explanation: Both pairs are opposites: 'long' vs 'short' (size opposites) and 'signed' vs 'unsigned' (sign opposites). Together they form the complete modifier system for integer types.

Q105. Which of the following stores the largest positive value?

(A) signed int (4 bytes)

(B) unsigned int (4 bytes)

(C) signed long (4 bytes)

(D) short (2 bytes)

✔ **Answer: (B)**

Explanation: unsigned int (4 bytes) holds up to 4,294,967,295. signed int (4 bytes) max is 2,147,483,647. unsigned int wins for maximum positive value with same byte count.

Q106. Among the following, which has the smallest storage size?

(A) short int

(B) int

(C) long int

(D) long long int

✔ **Answer: (A)**

Explanation: short int (2 bytes) < int (4 bytes) ≤ long int (4 or 8 bytes) < long long int (8 bytes). So short int has the smallest guaranteed size.

Q107. What is the difference in maximum values between 'unsigned int' and 'signed int' (4 bytes)?

(A) Equal

(B) Unsigned max is exactly double signed max + 1

(C) Signed max is larger

(D) Difference is 2^{31}

✔ **Answer: (B)**

Explanation: Signed int max = 2^{31}–1 = 2,147,483,647. Unsigned int max = 2^{32}–1 = 4,294,967,295. Ratio: 4,294,967,295 / 2,147,483,647 ≈ 2. Unsigned max ≈ 2×signed max + 1.

Q108. sizeof(signed int) compared to sizeof(unsigned int) is:

(A) signed is larger

(B) unsigned is larger

(C) They are always equal

(D) Depends on the value stored

✔ **Answer: (C)**

Explanation: sizeof() measures storage, not range. Both signed int and unsigned int occupy the same number of bytes. The modifier only changes how the bit pattern is interpreted.

Q109. Which declaration correctly defines a variable to hold the value 50,000?

(A) short int x = 50000; /* valid */

(B) signed short x = 50000; /* valid */

(C) unsigned short x = 50000; /* valid */

(D) signed char x = 50000; /* valid */

✔ **Answer: (C)**

Explanation: 50,000 is within [0, 65,535] (unsigned short range). signed short max is 32,767 — too small. char (1 byte) can't hold 50,000. unsigned short is the correct choice.

Q110. Which data type should be used to count file sizes up to several gigabytes?

(A) int

(B) unsigned int

(C) long long int

(D) short int

✔ **Answer: (C)**

Explanation: File sizes can easily exceed 4 GB. unsigned int max ≈ 4.3 GB (barely enough). long long int (8 bytes) supports up to ~9.2×10^{18} bytes — the correct choice for large file sizes.

Q111. Which of the following correctly represents all possible byte values (0–255)?

(A) char

(B) signed char

(C) unsigned char

(D) short

✔ **Answer: (C)**

Explanation: unsigned char (0–255) represents all byte values exactly. signed char (–128 to 127) misses 128–255. char is implementation-defined. unsigned char is the portable byte type in C.

Q112. The 'char' data type in C has a size of 1 byte by definition because:

(A) It is mandated by the CPU

(B) sizeof(char) = 1 is mandated by the C standard

(C) ASCII requires 1 byte

(D) Unicode requires 1 byte

✔ **Answer: (B)**

Explanation: sizeof(char) is always 1 by C standard definition. sizeof measures in units of char, so sizeof(char)=1 is tautologically true. All other sizes are relative to this.

Q113. Which combination provides the maximum integer range in standard C?

(A) signed long long int

(B) unsigned long long int

(C) long double

(D) unsigned long int

✔ **Answer: (B)**

Explanation: unsigned long long int (8 bytes) holds 0 to 18,446,744,073,709,551,615 — the largest range for a standard C integer type.

Q114. What is the output of the code? int a = 2147483647; a = a + 1; printf("%d", a);

(A) 2147483648

(B) –2147483648

(C) 0

(D) Undefined behaviour (likely –2147483648)

✔ **Answer: (D)**

Explanation: Signed integer overflow (INT_MAX + 1) is undefined behaviour in C. In practice on two's complement systems, the result wraps to –2,147,483,648, but relying on this is dangerous.

Q115. Which statement is TRUE about 'long long' vs 'long'?

(A) long long is always 16 bytes

(B) long long is always 8 bytes

(C) long long >= long in size

(D) They always have the same size

✔ **Answer: (C)**

Explanation: The C standard guarantees sizeof(long long) ≥ sizeof(long) ≥ sizeof(int). In practice, long long=8B always, long=4B (Windows) or 8B (Linux 64-bit).

Q116. Which of the following expressions evaluates to TRUE (non-zero) in C?

(A) (signed int)(–1) == (unsigned int)(–1)

(B) (signed int)(–1) > (unsigned int)(0)

(C) (unsigned int)(UINT_MAX + 1U) == 1U

(D) (short)(32768) == 32768

✔ **Answer: (A)**

Explanation: –1 signed and –1 converted to unsigned both produce the all-ones bit pattern. In comparison, the int is converted to unsigned, so both sides equal UINT_MAX → TRUE. (B) is false (signed→unsigned makes –1 very large). (C) is 0==1, false. (D): 32768 overflows short.

Q117. What will sizeof(unsigned long long) return on all conforming C99+ compilers?

(A) 4

(B) At least 8

(C) Exactly 8

(D) At least 4

✔ **Answer: (B)**

Explanation: The C99 standard guarantees sizeof(unsigned long long) ≥ 8. In practice it is exactly 8 bytes everywhere, but 'at least 8' is the technically correct guarantee.

Q118. Which type should be used for loop indices when iterating over an array with malloc'd size?

(A) int

(B) unsigned int

(C) size_t

(D) long

✔ **Answer: (C)**

Explanation: size_t is the correct type for array indices and memory sizes. It matches sizeof() and malloc() return type, avoids signed/unsigned comparison warnings, and is portable.

Q119. Which is the correct output of printf("%d", sizeof(int)); on most platforms?

(A) 2

(B) 4

(C) 8

(D) Compiler-dependent (technically %zu should be used)

✔ **Answer: (D)**

Explanation: sizeof returns size_t (unsigned). Using %d for size_t is technically undefined behaviour (though on 32/64-bit systems where sizeof(size_t)=sizeof(int), it prints 4). %zu is correct.

Q120. In C, which of the following will generate a signed/unsigned comparison warning?

(A) int i; size_t n; if(i < n)

(B) unsigned int x = 5; if(x < 10U)

(C) int a = –1; int b = 1; if(a < b)

(D) unsigned short x; unsigned short y; if(x < y)

✔ **Answer: (A)**

Explanation: Comparing a signed int with size_t (unsigned) generates the -Wsign-compare warning in GCC/Clang. The other comparisons involve same-sign types and don't trigger this.

Q121. What is the byte size hierarchy from smallest to largest for standard C integer types?

(A) char < short < int < long < long long

(B) short < char < int < long < long long

(C) char < short < long < int < long long

(D) All are 4 bytes

✔ **Answer: (A)**

Explanation: C standard guarantees: sizeof(char)=1 ≤ sizeof(short) ≤ sizeof(int) ≤ sizeof(long) ≤ sizeof(long long). The typical sizes are 1,2,4,4,8 bytes respectively.

Q122. Which of the following is used in embedded systems due to memory constraints?

(A) long long int

(B) double

(C) unsigned char and short int

(D) void*

✔ **Answer: (C)**

Explanation: Embedded systems (microcontrollers with KB of RAM) use compact types like unsigned char (1B) and short int (2B) to conserve memory. Long types are avoided when possible.

Q123. What is the range comparison? (Assume 4-byte int, 8-byte long long)

(A) Range of signed int > range of unsigned short

(B) Range of unsigned int = range of signed long long

(C) Range of unsigned long long > range of signed long long

(D) Range of signed short > range of unsigned char

✔ **Answer: (C)**

Explanation: unsigned long long: 0 to ~1.8×10^{19}. signed long long: ~-9.2×10^{18} to ~9.2×10^{18}. Total range of unsigned (count of values) = 2^{64} vs 2^{64} for signed — same count, but unsigned LL's positive range is twice as large.

Q124. Which modifier combination is needed to store the value 3,000,000,000 in C?

(A) int (signed)

(B) unsigned int

(C) short

(D) signed long

✔ **Answer: (B)**

Explanation: 3,000,000,000 > INT_MAX (2,147,483,647) but ≤ UINT_MAX (4,294,967,295). So unsigned int works. signed long (4B) also fails. unsigned int or long long are both valid; unsigned int is the minimal choice.

Q125. How many values can 'unsigned short int' (2 bytes) represent?

(A) 32,767

(B) 32,768

(C) 65,535

(D) 65,536

✔ **Answer: (D)**

Explanation: 2-byte unsigned short: 2^{16} = 65,536 distinct values (0 through 65,535 inclusive).

Chapter 7: Advanced & Tricky MCQs

Overflow, undefined behaviour, portability, type conversion

■ Questions 126–150 with Answers & Explanations

Q126. How many values can 'signed short int' (2 bytes) represent?

(A) 32,767

(B) 32,768

(C) 65,535

(D) 65,536

✔ Answer: (D)

Explanation: signed short int also represents 2^{16} = 65,536 distinct values (–32,768 through 32,767). The total count is the same as unsigned; only the range shifts.

Q127. The macro CHAR_BIT in specifies:

(A) The size of char in bytes

(B) The number of bits in a char

(C) The maximum char value

(D) The char encoding scheme

✔ Answer: (B)

Explanation: CHAR_BIT specifies the number of bits in a char (at minimum). On virtually all systems CHAR_BIT=8. The C standard requires CHAR_BIT ≥ 8.

Q128. Which type is guaranteed to be 32 bits exactly on any platform using ?

(A) int

(B) long

(C) int32_t

(D) unsigned int

✔ Answer: (C)

Explanation: int32_t (from , C99) is exactly 32 bits on any platform that supports it. Plain int/long sizes vary by platform. Use exact-width types for portability.

Q129. What is the output of: unsigned int x = 1; printf("%d", x << 31);

(A) 2147483648

(B) –2147483648

(C) 0

(D) Undefined behaviour

✔ Answer: (D)

Explanation: Shifting a 1-bit into the sign position of an unsigned type is well-defined (result = 2,147,483,648 = 2^{31}). However, printf with %d reinterprets unsigned as signed, which is implementation-defined. Strictly, %u should be used.

Q130. What is the output of: signed int x = 1; printf("%d", x << 31);

(A) 2147483648

(B) –2147483648

(C) 0

(D) Undefined behaviour

✔ **Answer: (D)**

Explanation: Left-shifting into or past the sign bit of a signed integer is undefined behaviour in C (prior to C20). Never shift signed integers into the sign bit.

Q131. Which type would you use to count bytes in a 1 TB file?

(A) unsigned int

(B) long

(C) unsigned long long

(D) double

✔ **Answer: (C)**

Explanation: 1 TB = 2^{40} bytes ≈ 1.1×10^{12}. UINT_MAX ≈ 4.3×10^{9} (insufficient). unsigned long long max ≈ 1.8×10^{19} — more than adequate.

Q132. What is the output of: printf("%d", (char)(-1));

(A) –1

(B) 255

(C) 0

(D) Compiler error

✔ **Answer: (A)**

Explanation: (char)(–1) on a signed-char platform: –1 is within signed char range (–128 to 127). When passed to printf and promoted to int, it remains –1. Output: –1.

Q133. Which of the following can store negative values without an explicit modifier?

(A) unsigned char

(B) unsigned int

(C) int

(D) unsigned short

✔ **Answer: (C)**

Explanation: Plain 'int' defaults to 'signed int' and can store negative values without any explicit modifier. All 'unsigned' prefixed types cannot store negatives.

Q134. In C, what is the result of adding INT_MAX + 1 to an unsigned int initialized to 0?

(A) INT_MAX + 1 = 2147483648 (stored correctly as unsigned)

(B) Overflow and wraps to negative

(C) 0

(D) Undefined behaviour

✔ **Answer: (A)**

Explanation: An unsigned int can hold 2,147,483,648 (within UINT_MAX = 4,294,967,295). The addition result is correct. Only UINT_MAX + 1 wraps to 0. No overflow here.

Q135. What is printed by: long x = 2147483648L; printf("%ld", x); (where long = 4 bytes)

(A) 2147483648

(B) –2147483648

(C) Undefined

(D) 0

✔ **Answer: (C)**

Explanation: 2,147,483,648 (2^{31}) exceeds LONG_MAX for a 4-byte long (2^{31}–1). The literal 2147483648L causes overflow, which is undefined behaviour (though practically wraps to –2,147,483,648).

Q136. What is wrong with: for(unsigned int i = 10; i >= 0; i--) { ... }

(A) Loop runs backwards, should use i++

(B) Infinite loop: unsigned can never be < 0, so i >= 0 is always true

(C) unsigned int can't be used in for loops

(D) i-- is illegal for unsigned

✔ **Answer: (B)**

Explanation: unsigned int i >= 0 is ALWAYS true (unsigned can't be negative). When i=0 and i-- executes, it wraps to UINT_MAX — creating an infinite loop. A classic C pitfall.

Q137. Which of the following correctly converts a signed int to unsigned without undefined behaviour?

(A) unsigned int u = (unsigned int)s;

(B) unsigned int u = abs(s);

(C) unsigned int u = –s;

(D) unsigned int u = s & 0x7FFFFFFF;

✔ **Answer: (A)**

Explanation: Casting a signed int to unsigned int is well-defined in C: the value is converted modulo 2^n. abs(s) gives the absolute value, not the same. Option D loses the sign bit's magnitude.

Q138. What is the correct way to check if an unsigned subtraction would underflow?

(A) if (a – b < 0)

(B) if (b > a) { /* would underflow */ }

(C) if (a – b == –1)

(D) Unsigned subtraction never underflows

✔ **Answer: (B)**

Explanation: For unsigned types, check if b > a BEFORE subtracting. 'if (a – b < 0)' never triggers since the result is unsigned (always ≥ 0). Option B is the correct guard.

Q139. What is 'integer promotion' in C?

(A) Casting int to long

(B) Converting char/short to int before arithmetic

(C) Making a variable unsigned

(D) Promoting float to double

✔ **Answer: (B)**

Explanation: Integer promotion: char, short, and bit-fields are automatically promoted to int (or unsigned int if the value doesn't fit int) before arithmetic operations. This prevents overflow in common

calculations.

Q140. What are 'usual arithmetic conversions' in C?

(A) Converting all types to float

(B) The rules C uses to bring operands of different types to a common type

(C) Converting int to char

(D) A compiler optimization

✔ **Answer: (B)**

Explanation: Usual arithmetic conversions: when operands have different types, C converts them to a common type following a hierarchy (long double > double > float > unsigned long long > … > int), ensuring meaningful arithmetic.

Q141. Which header defines ULONG_MAX, UINT_MAX, etc.?

(A)

(B)

(C)

(D)

✔ **Answer: (C)**

Explanation: defines all the limit macros for integer types: CHAR_MIN, CHAR_MAX, SHRT_MAX, INT_MAX, LONG_MAX, LLONG_MAX, UINT_MAX, ULONG_MAX, ULLONG_MAX, etc.

Q142. What is ptrdiff_t and why is it signed?

(A) An unsigned type for pointer storage

(B) A signed type for the difference between two pointers

(C) A type alias for int

(D) A macro for pointer size

✔ **Answer: (B)**

Explanation: ptrdiff_t (from) is a signed integer type for the difference between two pointers. It's signed because pointer1 – pointer2 can be negative (when pointer1 < pointer2).

Q143. Which of the following demonstrates a portable way to swap unsigned int bits?

(A) x = –x;

(B) x = ~x + 1;

(C) x ^= 0xFFFFFFFFU;

(D) x = !x;

✔ **Answer: (C)**

Explanation: x ^= 0xFFFFFFFFU performs a bitwise NOT on all 32 bits of unsigned x (XOR with all 1s flips every bit). ~x is equivalent but may be used on signed types where it's also well-defined.

Q144. What does the suffix 'U' accomplish in the literal '0xFFFFFFFFU'?

(A) Ensures the literal is treated as unsigned

(B) Makes it a union literal

(C) Adds unsigned extension bits

(D) No effect; all hex literals are unsigned

✔ **Answer: (A)**

Explanation: 'U' suffix ensures the literal is unsigned int. Without it, 0xFFFFFFFF might be treated as long (if it doesn't fit in int), which could cause unexpected results in comparisons and assignments.

Q145. Which data type is typically used to represent RGB colour components (0–255)?

(A) int

(B) signed char

(C) unsigned char

(D) short

✔ **Answer: (C)**

Explanation: RGB colour components range from 0 to 255, which fits exactly in unsigned char (8 bits, 0–255). This is the standard type used in image processing libraries.

Q146. What is the purpose of int8_t, uint16_t, etc. types?

(A) Faster computation

(B) Exact bit-width guarantee across all platforms

(C) Larger storage

(D) Platform-specific optimization

✔ **Answer: (B)**

Explanation: Exact-width types (int8_t, uint16_t, int32_t, uint64_t) from guarantee the specified width on any conforming platform, enabling portable bit-level code.

Q147. Which type should be used to represent a Unicode code point (U+0000 to U+10FFFF)?

(A) unsigned char (1B)

(B) unsigned short (2B)

(C) unsigned int (4B) or uint32_t

(D) long long

✔ **Answer: (C)**

Explanation: Unicode supports up to U+10FFFF = 1,114,111. unsigned short (max 65,535) is insufficient. unsigned int (max 4,294,967,295) or uint32_t comfortably holds all Unicode code points.

Q148. Why is 'unsigned int' preferred over 'signed int' for bitmask operations?

(A) Unsigned is faster on all CPUs

(B) Unsigned avoids undefined behaviour in shift operations

(C) Unsigned has more bits

(D) Signed cannot be used in bitwise operations

✔ **Answer: (B)**

Explanation: Shifting a signed integer by amounts that affect the sign bit is undefined behaviour. Unsigned types have well-defined bit shift behaviour, making them the standard choice for bitmasks and flags.

Q149. Which of the following declares an array that can hold exactly 1,000 values between 0 and 255 efficiently?

(A) int arr[1000];

(B) unsigned char arr[1000];

(C) long arr[1000];

(D) double arr[1000];

✔ **Answer: (B)**

Explanation: unsigned char (1 byte, 0–255) is the most memory-efficient choice for byte values. int would waste 3 bytes per element (4000 bytes total vs 1000 bytes).

Q150. What is the total memory occupied by: short arr[100]; on a typical system?

(A) 100 bytes

(B) 200 bytes

(C) 400 bytes

(D) 800 bytes

✔ **Answer: (B)**

Explanation: short int = 2 bytes. 100 elements × 2 bytes = 200 bytes total. Compared to int arr[100] = 400 bytes, using short saves 200 bytes.

Chapter MT: Mock Test Papers

Attempt under timed conditions — 45 minutes each

■ Mock Test 1: Fundamentals & long

Time: 45 minutes | Marks: 1 per question | No negative marking

Instructions: Choose the BEST answer. Cover the answer keys section.

Q1. Which of the following are type modifiers (qualifiers) in C language?

(A) long, short, signed, unsigned
(B) static, extern, register, auto
(C) const, volatile, restrict
(D) int, float, double, char

Q2. Which type modifier can be applied to the 'double' data type in C?

(A) short
(B) long
(C) unsigned
(D) Both A and C

Q3. What is the primary purpose of type modifiers in C?

(A) To change the type of a variable at runtime
(B) To modify the size and range of basic data types
(C) To provide type-casting functionality
(D) To define pointer types

Q4. Which of the following declarations is INVALID in C?

(A) unsigned int x;
(B) long double y;
(C) short float z;
(D) signed char c;

Q5. The keyword 'signed' in C is used to:

(A) Allow only positive values
(B) Allow both positive and negative values
(C) Double the storage size
(D) Make a variable constant

Q6. Which modifier doubles the range of positive values compared to 'signed'?

(A) long
(B) short
(C) unsigned
(D) extern

Q7. In C, the default sign of 'int' is:

(A) unsigned
(B) signed
(C) depends on the compiler
(D) not applicable

Q8. Which of the following is NOT a valid use of 'long' modifier?

(A) long int
(B) long double
(C) long char
(D) long long int

Q9. What is the size of 'short int' on most 32-bit and 64-bit systems?

(A) 1 byte
(B) 2 bytes
(C) 4 bytes
(D) 8 bytes

Q10. Which C standard guarantees that sizeof(short) ≤ sizeof(int) ≤ sizeof(long)?

(A) ANSI C (C89/C90)
(B) C99
(C) C11
(D) All C standards

Q11. Which modifier is used to extend the range of an integer by using more storage?

(A) short
(B) unsigned
(C) long
(D) register

Q12. Type modifier 'unsigned' can be applied to which of the following?

(A) float and double
(B) int and char only
(C) int, char, short, long
(D) All data types

Q13. Which of the following correctly declares a 'short unsigned integer'?

(A) short unsigned int x;
(B) unsigned short x;
(C) Both A and B
(D) Neither A nor B

Q14. What does the 'signed' keyword do when applied to 'char'?

(A) Makes char hold –128 to 127

(B) Makes char hold 0 to 255

(C) Makes char hold –256 to 255

(D) No effect on char

Q15. The 'unsigned char' type in C has a range of:

(A) –128 to 127

(B) 0 to 127

(C) 0 to 255

(D) –256 to 255

Q16. What is the minimum size (in bytes) guaranteed by the C standard for 'long long int'?

(A) 4

(B) 6

(C) 8

(D) 16

Q17. Which header file provides the exact-width integer types like int8_t, uint32_t?

(A)

(B)

(C)

(D)

Q18. Which macro in gives the maximum value of 'unsigned int'?

(A) INT_MAX

(B) UINT_MAX

(C) ULONG_MAX

(D) SHORT_MAX

Q19. Which of the following statements about type modifiers is TRUE?

(A) 'short' always means 2 bytes on all platforms

(B) 'long' always means 8 bytes on all platforms

(C) sizeof(int) is guaranteed to be 4 bytes

(D) sizeof(short) ≤ sizeof(long) is always guaranteed

Q20. When 'long' is written alone without 'int', it is treated as:

(A) long double

(B) long float

(C) long int

(D) Compiler error

Q21. What is the typical size of 'long int' on a 32-bit compiler?

(A) 2 bytes
(B) 4 bytes
(C) 8 bytes
(D) 16 bytes

Q22. What is the size of 'long int' on a 64-bit Linux system (GCC)?

(A) 4 bytes
(B) 6 bytes
(C) 8 bytes
(D) 16 bytes

Q23. What is the format specifier for 'long int' in C?

(A) %d
(B) %ld
(C) %li
(D) Both B and C

Q24. What is the range of 'long int' when it is 4 bytes?

(A) –32,768 to 32,767
(B) –2,147,483,648 to 2,147,483,647
(C) 0 to 4,294,967,295
(D) –9,223,372,036,854,775,808 to 9,223,372,036,854,775,807

Q25. What is the range of 'long int' when it is 8 bytes (64-bit)?

(A) 0 to 18,446,744,073,709,551,615
(B) –2,147,483,648 to 2,147,483,647
(C) –9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
(D) –4,294,967,296 to 4,294,967,295

■ Mock Test 2: short & signed

Time: 45 minutes | Marks: 1 per question | No negative marking

Instructions: Choose the BEST answer. Cover the answer keys section.

Q1. What does 'long long int' guarantee in terms of size?

(A) At least 4 bytes
(B) At least 6 bytes
(C) Exactly 8 bytes
(D) At least 8 bytes

Q2. Which format specifier is used for 'long long int'?

(A) %ld
(B) %llx
(C) %lld
(D) %Ld

Q3. What is the format specifier for 'unsigned long int'?

(A) %u
(B) %lu
(C) %ul
(D) %ld

Q4. Evaluate: What will sizeof(long long int) return on a 32-bit GCC compiler?

(A) 4
(B) 6
(C) 8
(D) 16

Q5. Which suffix is used to denote a long integer literal in C?

(A) L or l
(B) U or u
(C) LL or ll
(D) LU or lu

Q6. Which suffix is used to denote a long long integer literal?

(A) L
(B) ll
(C) LL or ll
(D) LLL

Q7. The LLONG_MAX macro represents the maximum value of 'long long int'. Its value is approximately:

(A) 2.1 billion
(B) 4.3 billion
(C) 9.2×10^{18}
(D) 1.8×10^{19}

Q8. Which C standard introduced 'long long int'?

(A) ANSI C (C89)

(B) C95

(C) C99

(D) C11

Q9. What is printed by: printf("%ld", 2147483648L); on a system where long is 4 bytes?

(A) 2147483648

(B) -2147483648

(C) 0

(D) Undefined behaviour

Q10. If sizeof(long) = 8, what is LONG_MAX?

(A) 2,147,483,647

(B) 4,294,967,295

(C) 9,223,372,036,854,775,807

(D) 18,446,744,073,709,551,615

Q11. Which of the following is the correct way to read a long int using scanf?

(A) scanf("%d", &x;);

(B) scanf("%ld", &x;);

(C) scanf("%i", &x;);

(D) scanf("%li", &x;);

Q12. Which data model uses long = 8 bytes and int = 4 bytes?

(A) ILP32

(B) LP64

(C) LLP64

(D) ILP64

Q13. In which data model is 'long' still 4 bytes even on a 64-bit OS?

(A) LP64

(B) ILP64

(C) LLP64 (Windows 64-bit)

(D) ILP32

Q14. What is the output of: printf("%d", sizeof(long)); on a typical 64-bit Linux?

(A) 2

(B) 4

(C) 8

(D) 16

Q15. Which is the correct portable way to print sizeof() results?

(A) %d
(B) %i
(C) %u
(D) %zu

Q16. A 'long double' typically provides how many bytes of storage in GCC on x86-64 Linux?

(A) 8 bytes
(B) 10 bytes
(C) 12 bytes
(D) 16 bytes (80-bit extended stored in 16B)

Q17. What is the size of 'short int' in C?

(A) 1 byte
(B) 2 bytes
(C) 4 bytes
(D) Depends only on OS

Q18. What is the range of 'signed short int' (2 bytes)?

(A) 0 to 65,535
(B) –128 to 127
(C) –32,768 to 32,767
(D) –2,147,483,648 to 2,147,483,647

Q19. What is the range of 'unsigned short int' (2 bytes)?

(A) –32,768 to 32,767
(B) 0 to 32,767
(C) 0 to 65,535
(D) 0 to 65,536

Q20. The format specifier for 'short int' in printf is:

(A) %d
(B) %hd
(C) %sd
(D) %i

Q21. The format specifier for 'unsigned short int' is:

(A) %u
(B) %hu
(C) %su
(D) %uh

Q22. Which macro holds the maximum value of 'short int'?

(A) SHORT_MAX
(B) SHRT_MAX
(C) MAX_SHORT
(D) INT16_MAX

Q23. Which macro holds the minimum value of 'short int'?

(A) SHRT_MIN
(B) SHORT_MIN
(C) INT16_MIN
(D) Both A and C

Q24. Why is 'short int' used instead of 'int' in some applications?

(A) It provides higher precision
(B) It saves memory in large arrays
(C) It is faster on all CPUs
(D) It allows negative values

Q25. What happens when a short int value exceeds 32,767 (e.g., you assign 32768)?

(A) It becomes 32768
(B) It wraps around to –32,768
(C) It becomes 0
(D) Runtime error

■ Mock Test 3: unsigned & Comparison

Time: 45 minutes | Marks: 1 per question | No negative marking

Instructions: Choose the BEST answer. Cover the answer keys section.

Q1. What is the result of: short s = 32767; s++; (assuming two's complement)?

(A) 32768
(B) –32768
(C) 0
(D) Undefined, then likely –32768

Q2. Which of the following is a valid declaration of a short integer?

(A) short x;
(B) short int x;
(C) int short x;
(D) All of the above

Q3. What is sizeof(short) + sizeof(int) on a typical 32-bit system?

(A) 4
(B) 6
(C) 8
(D) Compiler-dependent

Q4. In which scenario would 'short int' be preferred over 'int'?

(A) When you need values beyond 32,767
(B) When declaring a single counter variable
(C) When storing large arrays of small numbers to save memory
(D) When writing to a long-term file

Q5. What is the size of 'unsigned short' in bytes?

(A) 1
(B) 2
(C) 4
(D) Same as int

Q6. Which of the following represents a 16-bit port number correctly in C?

(A) int port = 8080;
(B) short port = 8080;
(C) unsigned short port = 8080;
(D) char port = 8080;

Q7. USHRT_MAX in is equal to:

(A) 32,767
(B) 65,535
(C) 65,536
(D) 32,768

Q8. When a short int is passed to printf with %d, what conversion occurs?

(A) Nothing, %d works directly

(B) Integer promotion to int happens automatically

(C) The value is truncated to 1 byte

(D) Undefined behaviour always occurs

Q9. What is the two's complement binary representation of –1 in a 16-bit short?

(A) 0111 1111 1111 1111

(B) 1000 0000 0000 0001

(C) 1111 1111 1111 1111

(D) 1000 0000 0000 0000

Q10. What is the binary representation of SHRT_MAX (32,767) in 16 bits?

(A) 0111 1111 1111 1111

(B) 1111 1111 1111 1111

(C) 0111 1111 0000 0000

(D) 1000 0000 0000 0000

Q11. What is SHRT_MIN, the minimum value of short int?

(A) 0

(B) –32,767

(C) –32,768

(D) –65,536

Q12. Which of the following is true about the 'signed' modifier?

(A) It is the default for all integer types

(B) It allows values from 0 to 2n-1

(C) It allows both positive and negative values

(D) It only applies to char

Q13. What is the range of 'signed char'?

(A) 0 to 255

(B) –128 to 127

(C) –127 to 127

(D) –256 to 255

Q14. The two's complement representation of –128 in a signed char (8 bits) is:

(A) 0111 1111

(B) 1111 1111

(C) 1000 0000

(D) 1000 0001

Q15. What is the output of the following code? signed int x = -1; printf("%u", x);

(A) –1

(B) 4294967295

(C) 0

(D) Compiler error

Q16. For signed integer types, how is negative zero handled in C?

(A) It is a valid value distinct from zero

(B) Two's complement has no negative zero

(C) It is defined as –0 in the standard

(D) It causes undefined behaviour

Q17. CHAR_MIN in will be –128 if:

(A) char is unsigned

(B) char is signed

(C) char is 16-bit

(D) char is long

Q18. Which of the following produces correct results when checking if c is a valid ASCII character?

(A) if (c >= 0 && c < 128) — assuming char is signed

(B) if ((unsigned char)c < 128)

(C) Both A and B are correct

(D) Neither is correct

Q19. What will happen if –129 is assigned to a signed char?

(A) The value is stored as –129

(B) Wraps around: –129 becomes 127

(C) Wraps around: –129 becomes –1

(D) Undefined/implementation-defined overflow

Q20. What does 'int' default to if no sign modifier is written?

(A) unsigned

(B) signed

(C) depends on the value assigned

(D) compiler chooses randomly

Q21. Which bit is the sign bit in a signed representation?

(A) Least Significant Bit (LSB)

(B) Most Significant Bit (MSB)

(C) Middle bit

(D) Second LSB

Q22. The expression (signed char)(200) on a typical system gives:

(A) 200

(B) –56

(C) 56

(D) 0

Q23. What is the value of INT_MAX on a system where sizeof(int) = 4?

(A) 32,767

(B) 65,535

(C) 2,147,483,647

(D) 4,294,967,295

Q24. What is INT_MIN on a 4-byte signed int system?

(A) –2,147,483,647

(B) –2,147,483,648

(C) –4,294,967,295

(D) –2,147,483,649

Q25. Which of the following will cause signed integer overflow (undefined behaviour)?

(A) int x = INT_MAX; x++;

(B) unsigned int x = UINT_MAX; x++;

(C) short x = 32767; x = (short)(x+1);

(D) Both A and C

■ Mock Test 4: Full Mixed Paper

Time: 45 minutes | Marks: 1 per question | No negative marking

Instructions: Choose the BEST answer. Cover the answer keys section.

Q1. Which of the following are type modifiers (qualifiers) in C language?

(A) long, short, signed, unsigned
(B) static, extern, register, auto
(C) const, volatile, restrict
(D) int, float, double, char

Q2. Which modifier doubles the range of positive values compared to 'signed'?

(A) long
(B) short
(C) unsigned
(D) extern

Q3. Which modifier is used to extend the range of an integer by using more storage?

(A) short
(B) unsigned
(C) long
(D) register

Q4. What is the minimum size (in bytes) guaranteed by the C standard for 'long long int'?

(A) 4
(B) 6
(C) 8
(D) 16

Q5. What is the typical size of 'long int' on a 32-bit compiler?

(A) 2 bytes
(B) 4 bytes
(C) 8 bytes
(D) 16 bytes

Q6. What does 'long long int' guarantee in terms of size?

(A) At least 4 bytes
(B) At least 6 bytes
(C) Exactly 8 bytes
(D) At least 8 bytes

Q7. Which suffix is used to denote a long long integer literal?

(A) L
(B) ll
(C) LL or ll
(D) LLL

Q8. Which of the following is the correct way to read a long int using scanf?

(A) scanf("%d", &x;);

(B) scanf("%ld", &x;);

(C) scanf("%i", &x;);

(D) scanf("%li", &x;);

Q9. A 'long double' typically provides how many bytes of storage in GCC on x86-64 Linux?

(A) 8 bytes

(B) 10 bytes

(C) 12 bytes

(D) 16 bytes (80-bit extended stored in 16B)

Q10. The format specifier for 'unsigned short int' is:

(A) %u

(B) %hu

(C) %su

(D) %uh

Q11. What is the result of: short s = 32767; s++; (assuming two's complement)?

(A) 32768

(B) –32768

(C) 0

(D) Undefined, then likely –32768

Q12. Which of the following represents a 16-bit port number correctly in C?

(A) int port = 8080;

(B) short port = 8080;

(C) unsigned short port = 8080;

(D) char port = 8080;

Q13. What is SHRT_MIN, the minimum value of short int?

(A) 0

(B) –32,767

(C) –32,768

(D) –65,536

Q14. For signed integer types, how is negative zero handled in C?

(A) It is a valid value distinct from zero

(B) Two's complement has no negative zero

(C) It is defined as –0 in the standard

(D) It causes undefined behaviour

Q15. Which bit is the sign bit in a signed representation?

(A) Least Significant Bit (LSB)

(B) Most Significant Bit (MSB)

(C) Middle bit

(D) Second LSB

Q16. What arithmetic is used to represent negative numbers in virtually all modern CPUs?

(A) Sign-magnitude

(B) One's complement

(C) Two's complement

(D) BCD

Q17. Which header provides the abs() function for signed int?

(A)

(B)

(C)

(D)

Q18. What is the value of UINT_MAX on a 4-byte unsigned int system?

(A) 2,147,483,647

(B) 4,294,967,295

(C) 4,294,967,296

(D) 2,147,483,648

Q19. Which of the following is used to store a memory address or object size safely?

(A) unsigned int

(B) int

(C) size_t

(D) long int

Q20. In a comparison 'if (a > b)' where a is signed int (–1) and b is unsigned int (0), what happens?

(A) –1 > 0 is false, as expected

(B) –1 is converted to unsigned (4294967295), so 4294967295 > 0 is true

(C) Compiler rejects mixed comparison

(D) Both are converted to float

Q21. The format specifier for 'unsigned long long int' is:

(A) %lu

(B) %llu

(C) %uLL

(D) %Llu

Q22. Among the following, which has the smallest storage size?

(A) short int
(B) int
(C) long int
(D) long long int

Q23. Which of the following correctly represents all possible byte values (0–255)?

(A) char
(B) signed char
(C) unsigned char
(D) short

Q24. Which of the following expressions evaluates to TRUE (non-zero) in C?

(A) (signed int)(–1) == (unsigned int)(–1)
(B) (signed int)(–1) > (unsigned int)(0)
(C) (unsigned int)(UINT_MAX + 1U) == 1U
(D) (short)(32768) == 32768

Q25. What is the byte size hierarchy from smallest to largest for standard C integer types?

(A) char < short < int < long < long long
(B) short < char < int < long < long long
(C) char < short < long < int < long long
(D) All are 4 bytes

Q26. How many values can 'signed short int' (2 bytes) represent?

(A) 32,767
(B) 32,768
(C) 65,535
(D) 65,536

Q27. Which type would you use to count bytes in a 1 TB file?

(A) unsigned int
(B) long
(C) unsigned long long
(D) double

Q28. What is wrong with: for(unsigned int i = 10; i >= 0; i--) { ... }

(A) Loop runs backwards, should use i++
(B) Infinite loop: unsigned can never be < 0, so i >= 0 is always true
(C) unsigned int can't be used in for loops
(D) i-- is illegal for unsigned

Q29. Which header defines ULONG_MAX, UINT_MAX, etc.?

(A)

(B)

(C)

(D)

Q30. What is the purpose of int8_t, uint16_t, etc. types?

(A) Faster computation

(B) Exact bit-width guarantee across all platforms

(C) Larger storage

(D) Platform-specific optimization

Q31. What is the total memory occupied by: short arr[100]; on a typical system?

(A) 100 bytes

(B) 200 bytes

(C) 400 bytes

(D) 800 bytes

Chapter AK: Answer Keys

For all Mock Tests

■ Answer Key: Mock Test 1: Fundamentals & long

Q1: **(A)**	Q2: **(B)**	Q3: **(B)**	Q4: **(C)**	Q5: **(B)**	Q6: **(C)**	Q7: **(B)**	Q8: **(C)**
Q9: **(B)**	Q10: **(D)**	Q11: **(C)**	Q12: **(C)**	Q13: **(C)**	Q14: **(A)**	Q15: **(C)**	Q16: **(C)**
Q17: **(C)**	Q18: **(B)**	Q19: **(D)**	Q20: **(C)**	Q21: **(B)**	Q22: **(C)**	Q23: **(D)**	Q24: **(B)**
Q25: **(C)**							

■ Answer Key: Mock Test 2: short & signed

Q1: **(D)**	Q2: **(C)**	Q3: **(B)**	Q4: **(C)**	Q5: **(A)**	Q6: **(C)**	Q7: **(C)**	Q8: **(C)**
Q9: **(D)**	Q10: **(C)**	Q11: **(B)**	Q12: **(B)**	Q13: **(C)**	Q14: **(C)**	Q15: **(D)**	Q16: **(D)**
Q17: **(B)**	Q18: **(C)**	Q19: **(C)**	Q20: **(B)**	Q21: **(B)**	Q22: **(B)**	Q23: **(D)**	Q24: **(B)**
Q25: **(B)**							

■ Answer Key: Mock Test 3: unsigned & Comparison

Q1: **(D)**	Q2: **(D)**	Q3: **(B)**	Q4: **(C)**	Q5: **(B)**	Q6: **(C)**	Q7: **(B)**	Q8: **(B)**
Q9: **(C)**	Q10: **(A)**	Q11: **(C)**	Q12: **(C)**	Q13: **(B)**	Q14: **(C)**	Q15: **(B)**	Q16: **(B)**
Q17: **(B)**	Q18: **(C)**	Q19: **(D)**	Q20: **(B)**	Q21: **(B)**	Q22: **(B)**	Q23: **(C)**	Q24: **(B)**
Q25: **(A)**							

■ Answer Key: Mock Test 4: Full Mixed Paper

Q1: **(A)**	Q2: **(C)**	Q3: **(C)**	Q4: **(C)**	Q5: **(B)**	Q6: **(D)**	Q7: **(C)**	Q8: **(B)**
Q9: **(D)**	Q10: **(B)**	Q11: **(D)**	Q12: **(C)**	Q13: **(C)**	Q14: **(B)**	Q15: **(B)**	Q16: **(C)**
Q17: **(B)**	Q18: **(B)**	Q19: **(C)**	Q20: **(B)**	Q21: **(B)**	Q22: **(A)**	Q23: **(C)**	Q24: **(A)**
Q25: **(A)**	Q26: **(D)**	Q27: **(C)**	Q28: **(B)**	Q29: **(C)**	Q30: **(B)**	Q31: **(B)**	

Chapter : Exam Tips & Key Formulas

Last-Minute Revision

Key Formula: Number of values for n-bit type

Total values = 2■ Signed range: –2■■¹ to 2■■¹ – 1 Unsigned range: 0 to 2■ – 1

Format Specifiers Quick Table

short: %hd | int: %d | long: %ld | long long: %lld unsigned int: %u | unsigned long: %lu | unsigned long long: %llu | size_t: %zu

Common Trap: Unsigned loop termination

for(unsigned i = n; i >= 0; i--) is an INFINITE LOOP Unsigned can NEVER be negative — i >= 0 is always true!

Common Trap: Signed/Unsigned comparison

Comparing signed int (–1) with unsigned int promotes –1 to UINT_MAX. Always cast explicitly when mixing signed and unsigned.

Sizes to memorise

char=1B | short=2B | int=4B | long=4B(Win)/8B(Linux) | long long=8B float=4B | double=8B | long double=10/16B

Limits to memorise

CHAR_MAX=127 | UCHAR_MAX=255 | SHRT_MAX=32767 | USHRT_MAX=65535 INT_MAX=2147483647 | UINT_MAX=4294967295 | LLONG_MAX≈9.2×10¹■